41039 Programming 1

Revision of Week 8: Inheritance and interfaces



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

How the revision session works

A recap before the next lecture

Write an answer before the reveal. Explain the state change, rule or contract that makes it follow. These are practice questions, with no new assessment requirements.

Part 1: on your own

- 6 short questions
- About 90 seconds per question
- Java and Python; name the language first

Part 2: teams of 3 or 4

- Draw the model
- Spot the errors
- Predict the output
- Write and test the code

A result needs a reason. A repair needs a counterexample.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

1. Overriding chooses the object method

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Animal pet = new Dog("Milo");
System.out.println(pet.speak());
class Animal {
    protected final String name;
    Animal(String name) { this.name=name; }
    String speak() { return name+" is quiet"; }
}
class Dog extends Animal {
    Dog(String name) { super(name); }
    @Override String speak() { return name+" says woof";
      }
}
```

Your answer

What is the complete output? Which line or rule decides it?

1. Overriding chooses the object method

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Animal pet = new Dog("Milo");
System.out.println(pet.speak());
class Animal {
    protected final String name;
    Animal(String name) { this.name=name; }
    String speak() { return name+" is quiet"; }
}
class Dog extends Animal {
    Dog(String name) { super(name); }
    @Override String speak() { return name+" says woof";
      }
}
```

Answer

```
Milo says woof
```

The variable type permits speak, while the object type selects Dog.speak at runtime.

2. Calling the parent in Python

Python 3; commit to an outcome before the reveal.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return self.name + " is quiet"
class Dog(Animal):
    def speak(self):
        return super().speak() + "; then woof"
print(Dog("Milo").speak())
```

Your answer

What is the complete output? Which line or rule decides it?

2. Calling the parent in Python

Python 3; commit to an outcome before the reveal.

```python
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return self.name + " is quiet"
class Dog(Animal):
    def speak(self):
        return super().speak() + "; then woof"
print(Dog("Milo").speak())
```

Answer

```
Milo is quiet; then woof
```

Dog extends the behaviour by calling the inherited implementation first.

3. Fields and methods are resolved differently

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Parent value = new Child();
System.out.println(value.label);
System.out.println(value.describe());
class Parent {
    String label="parent field";
    String describe() { return "parent method"; }
}
class Child extends Parent {
    String label="child field";
    @Override String describe() { return "child method";
      }
}
```

Your answer

What is the complete output? Which line or rule decides it?

3. Fields and methods are resolved differently

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Parent value = new Child();
System.out.println(value.label);
System.out.println(value.describe());
class Parent {
    String label="parent field";
    String describe() { return "parent method"; }
}
class Child extends Parent {
    String label="child field";
    @Override String describe() { return "child method";
      }
}
```

Answer

```
parent field
child method
```

The field name is selected from Parent, but the overridden instance method comes from Child.

4. A Python abstract base class

Python 3; commit to an outcome before the reveal.

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side * self.side
shape = Square(3)
print(shape.area())
print(isinstance(shape, Shape))
```

Your answer

What is the complete output? Which line or rule decides it?

4. A Python abstract base class

Python 3; commit to an outcome before the reveal.

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side * self.side
shape = Square(3)
print(shape.area())
print(isinstance(shape, Shape))
```

Answer

```
9
True
```

Square implements area and is a subtype of Shape; the area is 3×3.

5. An interface is a useful promise

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Shape[] shapes = {new Square(3), new Rectangle(2,4)};
for(Shape shape:shapes) System.out.println(shape.area())
    ;
interface Shape { int area(); }
class Square implements Shape {
    private final int side;
    Square(int side) { this.side=side; }
    public int area() { return side*side; }
}
class Rectangle implements Shape {
    private final int w,h;
    Rectangle(int w,int h) { this.w=w; this.h=h; }
    public int area() { return w*h; }
}
```

Your answer

What is the complete output? Which line or rule decides it?

5. An interface is a useful promise

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Shape[] shapes = {new Square(3), new Rectangle(2,4)};
for(Shape shape:shapes) System.out.println(shape.area())
    ;
interface Shape { int area(); }
class Square implements Shape {
    private final int side;
    Square(int side) { this.side=side; }
    public int area() { return side*side; }
}
class Rectangle implements Shape {
    private final int w,h;
    Rectangle(int w,int h) { this.w=w; this.h=h; }
    public int area() { return w*h; }
}
```

Answer

```
9
8
```

Square computes 9, Rectangle computes 8, and the caller needs only the common interface.

6. Duck typing uses a behavioural contract

Python 3; commit to an outcome before the reveal.

```
class Email:
    def send(self, text):
        return "email: " + text
class Screen:
    def send(self, text):
        return "screen: " + text
def notify(channel):
    print(channel.send("Hello"))
notify(Email())
notify(Screen())
```

Your answer

What is the complete output? Which line or rule decides it?

6. Duck typing uses a behavioural contract

Python 3; commit to an outcome before the reveal.

```
class Email:
    def send(self, text):
        return "email: " + text
class Screen:
    def send(self, text):
        return "screen: " + text
def notify(channel):
    print(channel.send("Hello"))
notify(Email())
notify(Screen())
```

Answer

```
email: Hello
screen: Hello
```

Each object implements send; Python calls the method available on the received object.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Form teams

- Teams of 3 or 4. One shared sheet, one spokesperson.
- For each challenge, record the result and the reason.
- Rotate the spokesperson. Other teams can challenge an explanation.
- The final task has starter files and a Java/Python solution pair.

Discuss first. Commit to an answer. Then compare with the reveal.

Draw it: Declared type, object type and an alias

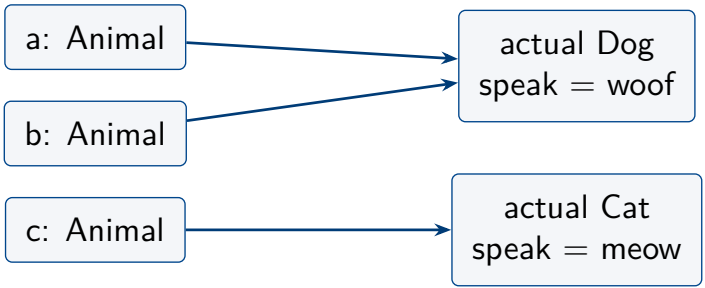
```
Animal a=new Dog();
Animal b=a;
Animal c=new Cat();
System.out.println(a.speak()+" "+b.speak()+" "+c.speak()
    );
System.out.println(a==b);
class Animal { String speak() { return "animal"; } }
class Dog extends Animal { String speak() { return "woof
    "; } }
class Cat extends Animal { String speak() { return "meow
    "; } }
```

Course website: Week 8. Complete programs accompany this deck.

Four minutes

Draw the object references after the shown operations. Label the important state. Mark what is shared, what changes, or which path handles failure. Predict every output line.

Draw it: one answer



Output

```
woof woof meow
true
```

The declared type permits speak(). Each actual object selects the overridden instance method. b aliases a, so no third animal is created.

Course website: Week 8. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Spot the errors 1: Java (5 min)

Contract

Through an Animal reference, the Dog named Milo must say Dog Milo.

```java
public class Broken {
    public static void main(String[] args) {
        Animal pet=new Dog("Milo");
        System.out.println(pet.speak());
    }
}
class Animal {
    protected final String name;
    Animal(String name) { this.name=name; }
    String speak() { return "Animal "+name; }
}
class Dog extends Animal {
    Dog(String name) { super("unknown"); }
    String speak(int unused) { return "Dog "+name; }
}
```

Spot the errors 1: answers

The repairs

- **L13:** Pass the provided name to the superclass constructor.
- **L14:** This overload does not override speak(). Use the same parameter list and add @Override to check it.

Course website: Week 8. Complete programs accompany this deck.

✗ Original program

```
Animal unknown
```

✓ After the repairs

```
Dog Milo
```

Spot the errors 2: Python (5 min)

Contract

Student("Linh").greet() must return Hello Linh! while retaining its mark.

```
1 class Person:
2     def __init__(self,name): self.name=
       name
3     def greet(self): return "Hello "+self.
       name
4 class Student(Person):
5     def __init__(self,name): self.mark=85
6     def greet(self): return super().greet+"
       !"
7
8 s=Student("Linh")
9 print(s.greet())
```

Spot the errors 2: answers

The repairs

- **L5:** Initialise the Person part with super().__init__(name).
- **L6:** Call the inherited method before concatenating its returned string.

✗ Original program

```
(no standard output)
TypeError: unsupported operand
    type(s) for +: 'method' and
    'str'
```

✓ After the repairs

```
Hello Linh!
```

Course website: Week 8. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Predict 1: Fields beside dynamic methods

```
Parent x=new Child();
System.out.println(x.label+" "+x.name());
Parent y=x;
System.out.println(y.name());
class Parent {
    String label="parent";
    String name() { return "P"; }
}
class Child extends Parent {
    String label="child";
    @Override String name() { return "C"; }
}
```

Course website: Week 8. Complete programs accompany this deck.

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Predict 1: answer

```
parent C
C
```

Why

Field access uses the declared Parent type; the overridden instance method uses the actual Child object. Assigning another reference changes neither.

Course website: Week 8. Complete programs accompany this deck.

Predict 2: Following super through a hierarchy

```
class A:
    def label(self): return "A"
class B(A):
    def label(self): return super().label()+"B"
class C(B):
    def label(self): return super().label()+"C"
print(C().label())
print(isinstance(C(),A))
```

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Course website: Week 8. Complete programs accompany this deck.

Predict 2: answer

```
ABC
True
```

Why

C calls the next implementation B, which calls A. The strings are extended while calls return. A C is also an A.

Course website: Week 8. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Code it: specification

One area contract, two shapes

Read S side or R width height using positive integer dimensions. Implement Square and Rectangle behind a shared area contract. Print the integer area. Explain why the caller does not need a shape-specific branch after construction.

Example input

```
S 3
```

Expected output

```
9
```

Eight minutes. One Java team and one Python team at the keyboard. Other teams produce a boundary test and explain which wrong implementation it would expose.

Code it: Java solution (1/2)

```
1 import java.util.Scanner;
2 Scanner sc=new Scanner(System.in);
3 String kind=sc.next();
4 Shape shape;
5 if(kind.equals("S")) shape=new Square(sc.nextInt());
6 else shape=new Rectangle(sc.nextInt(),sc.nextInt());
7 System.out.println(shape.area());
8 interface Shape { int area(); }
9 class Square implements Shape {
```

Course website: Week 8. Complete programs accompany this deck.

Code it: Java solution (2/2)

```
10      private final int s;
11      Square(int s) { this.s=s; }
12      public int area() { return s*s; }
13  }
14  class Rectangle implements Shape {
15      private final int w,h;
16      Rectangle(int w,int h) { this.w=w; this.h=h; }
17      public int area() { return w*h; }
18  }
```

Course website: Week 8. Complete programs accompany this deck.

Code it: Python solution

```python
class Shape:
    def area(self): raise NotImplementedError
class Square(Shape):
    def __init__(self,s): self.s=s
    def area(self): return self.s*self.s
class Rectangle(Shape):
    def __init__(self,w,h): self.w,self.h=w,h
    def area(self): return self.w*self.h
parts=input().split()
shape=Square(int(parts[1])) if parts[0]=="S" else Rectangle(int(parts[1]),int(parts[2]))
print(shape.area())
```

Course website: Week 8. Complete programs accompany this deck.

Code it: the tests

A slash separates input lines; (blank) denotes an empty line.

Test 1
`S 3`
Output: `9`

Test 2
`R 2 4`
Output: `8`

Test 3
`S 1`
Output: `1`

Explain why each test is present, not only whether it passes.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

What you should now be able to explain

- Trace declared types separately from actual object types.
- Distinguish overriding, overloading and field hiding.
- Initialise the inherited part of an object.
- Use an interface or duck-typed capability to keep callers general.

Return to the matching week page for runnable examples and further practice. Bring one unresolved prediction or counterexample to the next class.

Course website: Week 8. Complete programs accompany this deck.